

Technical Report: Gesture Recognition Using Inertial Sensors

Athanasios Sioutas (AM: 1100711)

email: up1100711@ac.upatras.gr Tel: +30 6944027375

Dorothea Velissaridi (AM: 1093332)

email: up1093332@ac.upatras.gr Tel: +30 6983661423

Department of Computer Engineering and Informatics, University of Patras

May 2026

1 Data Collection, Annotation & Controlled Environment

For this project we collected IMU data for five smartphone gestures: *scroll-down-index*, *scroll-up-index*, *swipe-left-index*, *swipe-right-index*, and *texting-single*. All movements were performed using only the right hand and the index finger, to match how people actually interact with a phone while scrolling through social media. Both members of our team acted as subjects (users 01 and 02).

Hardware & sensor placement. We used the Mbientlab MetaMotionR wristband, which embeds a Bosch BMI160 IMU. The sensor was worn on the right wrist for every session, with the watch face positioned at the upper-left corner of the wrist. We kept this placement identical for both subjects so that the coordinate frame would be consistent across recordings. During data collection, the wrist rested flat on a desk — this helped keep the arm stable and avoid introducing extra noise from involuntary arm movement. The actual gestures were performed on a smartphone, driven solely by the index finger.

Session structure. We split each gesture class into 5 independent recording sessions. Each session lasted 1 minute at a sampling rate of 100 Hz, giving roughly 6,000 samples per session and around 30,000 per gesture class per user. In total our dataset contains approximately 300,000 IMU frames across both subjects, five gesture classes, and six sensor axes (3-axis accelerometer + 3-axis gyroscope).

Annotation. We annotated the data at recording time — each document inserted into MongoDB already contained the gesture label (*gesture_id*), the subject ID (*user*), and the session number, which was tracked automatically as a counter per user-gesture pair. This way we avoided any manual post-hoc labelling and the mistakes that come with it.

2 Experimental Framework & Raw Windowed Paradigm

One of the first decisions we had to make was how to split the data into train and test sets. We initially tried a user-based split (train on one user, test on the other), but this gave poor results because the two subjects perform the same gesture differently in terms of speed and angle. A simple random window shuffle was also not acceptable, since overlapping windows from the same recording session would leak into both splits. We settled on a **session-based split**: sessions 1–4 go to training, session 5 is held out for testing. This way the model sees both users during training while the test set remains completely unseen. For preprocessing, we applied Z-score outlier removal (threshold = 3), linear interpolation for flagged samples, and a 4th-order Butterworth low-pass filter ($W_n = 0.1$) to clean up the signal. Segmentation used a sliding window of 500 samples with a 250-sample hop and a Hann weighting function.

In the first approach we simply flattened the raw windowed segments and fed them directly to the classifiers. The baseline results were 77.73% for RandomForest, 80.84% for SVM, and 78.74% for KNN. The most obvious problem we noticed was with vertical gestures — *scroll-up-index* had a recall of only 0.42, because its raw waveform looks almost identical to *scroll-down-index* when you look at the raw amplitude alone. Horizontal swipes were much easier to classify because they produce a clear sharp peak on the lateral axis. After running a Grid Search with 5-fold CV, RandomForest improved to 82.85%, SVM (RBF, $C = 10$) to 79.94%, and KNN to 79.74%.

3 Handcrafted Feature Engineering Pipeline

Since raw windows clearly struggled with similar-looking gestures, we moved to a proper feature engineering approach. For each of the six sensor axes we computed seven time-domain statistics (mean, std, min, max,

Table 1: Raw Windowed Baseline Classification Performance

Gesture Class	Precision	Recall	F1-Score	Support
scroll-down-index	0.8200	0.8200	0.8200	195
scroll-up-index	0.7500	0.4200	0.5400	196
swipe-left-index	0.8200	0.9700	0.8900	196
swipe-right-index	0.9300	0.9600	0.9400	226
texting-single	0.7700	0.9500	0.8500	184

median, range, zero-crossing rate) and three spectral features using FFT (spectral energy, dominant frequency, spectral entropy). On top of that we added two cross-channel metrics — Vector Magnitude (VM) and Signal Magnitude Area (SMA) — to make the feature set more robust to slight variations in wrist orientation. This gave us a **63-dimensional feature vector** per window.

We then applied a two-step feature selection process on the training data only: first, a Pearson correlation filter dropped 39 features with $|r| > 0.95$ to remove redundant dimensions; then, RandomForest feature importances were used to keep only the top 15 most useful features. Reducing the space from 63 to 15 features turned out to be very effective — accuracy jumped to **84.15 % for RandomForest**, 83.05 % for SVM, and 81.54 % for KNN. We also tried PCA (keeping 95 % of the variance) as an alternative dimensionality reduction method. It worked reasonably well for SVM (79.74 %) and KNN (78.64 %), but caused a notable drop for RandomForest (68.81 %). This makes sense because PCA produces abstract linear combinations of the original features, which removes the physical meaning that decision trees rely on when choosing split points.

Table 2: Comprehensive Performance Matrix Across Pipelines

Pipeline Configuration	RandomForest	SVM	KNN
Approach 1: Raw Baseline	0.7773	0.8084	0.7874
Approach 1: Raw Tuned (Grid)	0.8285	0.7994	0.7974
Approach 2: Feature Selection (Top 15)	0.8415	0.8305	0.8154
Approach 2: PCA (95% Variance)	0.6881	0.7974	0.7864

Grid Search hyperparameter tuning was also run on the optimised 15-feature space (Table 3). A clear difference is visible between the internal cross-validation scores (77–78 %) and the higher final test accuracies. This is expected: 5-fold CV trains the models on only 80 % of the data per fold, producing a more conservative metric. When the tuned parameters are applied to 100 % of the training split, the models generalise much better on the independent Session 5 test set.

Table 3: GridSearchCV Results on Optimized Feature Space

Model	Optimal Hyperparameters	CV Best Score
RandomForest	{max_depth: None, n_estimators: 200}	0.7769
SVM	{C: 10, kernel: ‘rbf’}	0.7898
KNN	{n_neighbors: 3, weights: ‘uniform’}	0.7798

Conclusion. One thing worth noting is the gap between cross-validation scores (roughly 77–79 %) and the final test accuracies. This is actually expected: during 5-fold CV the model only trains on 80 % of the available training sessions per fold, so the score is more conservative. When we retrain on the full training split and evaluate on Session 5, the models generalise noticeably better. Overall, the feature engineering pipeline clearly outperformed feeding raw windows directly to the classifiers. Compressing the data down to just 15 well-chosen features gave us better accuracy and a much lighter model — which is exactly what you want for a low-power edge AIoT application like this one.

Each notebook in the project codebase includes detailed step-by-step documentation (in the form of Markdown cells) describing the exact procedures followed at every stage of the pipeline — from data collection and MongoDB ingestion, to preprocessing, segmentation, feature extraction, and model evaluation.